

Filluminati

They know what you're going to type... before you do.

What is tab completion?

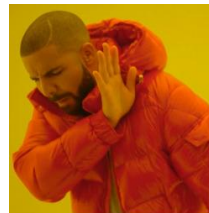
Demo??

```
def numberOfIslands(grid):  
    rl = len(grid)  
    cl = len(grid[0])  
    islands = 0  
    visited = set()  
  
    def dfs(row, col):  
        if ~
```

You, 17 seconds ago • Uncommitted changes

My Plan

- Give an LLM code around cursor as context and prompt it to autocomplete
- Use the smallest model possible.
- These ideas were kind of right but not entirely



Writing
code
manually



Tab
completion
go brrr

Reality Check #1: Context Format Matters More Than You Think

- Single line context
- Around 5 lines of prefix and 2 lines of suffix
- You must be thinking just give it more context that would solve it.
- Giving an LLM more context without structure is like giving your friend directions by reading them the entire map
- Using AST parsing and the LSP to get relevant code snippets from different files.



```
def numberOfIslands(grid):  
    rl = len(grid)  
    cl = len(grid[0])  
    islands = 0  
    visited = set()  
  
    def dfs(row, col):  
        if
```

You, 2 minutes ago • Uncommitted c

```
def numberOfIslands(grid):
```

```
    rl = len(grid)
```

```
    cl = len(grid[0])
```

```
    islands = 0
```

```
    visited = set()
```

```
    def dfs(row, col):
```

Reality Check #2: The Model Plateau

- Tried using Gemini, StarCoder, Qwen2.5 before going with Zeta
- Fill-in-the-Middle (FIM) prompt structure

FIM Prompt

```
<fim_prefix>
def process_data(items):
    results = []
    for item in items:
<fim_suffix>

results.append(processed)
    return results
<fim_middle>
```

Zeta Prompt

```
@src/quicksort.rs
use std::cmp::Ord;
<|editable_region_start|>
    pub fn quicksort<T:
Ord>(arr: &mut [T]) {
    let len = arr.len();
    if len <= 1 {
        return;
    }
    let pivot = partit
<|user_cursor_is_here|>
<|editable_region_end|>
```

```
def topological_sort(graph):
    from collections import defaultdict, deque

    in_degree = defaultdict(int)
    adj_list = defaultdict(list)

    for node in graph:
        if node:
            in_degree[node] = 0
            for neighbor in graph[node]:
                adj_list[neighbor].append(node)
            in_degree[node] += 1
    queue = deque([node for node in graph if in_degree[node] == 0])
    result = []
    while queue:
        current = queue.popleft()
        result.append(current)
        for neighbor in adj_list[current]:
            in_degree[neighbor] -= 1
            if in_degree[neighbor] == 0:
                queue.append(neighbor)

    if len(result) != len(graph):
        return None # Cycle detected

    return result
```

**NO
CONTEXT**



**SOME
CONTEXT**



**MORE
CONTEXT**



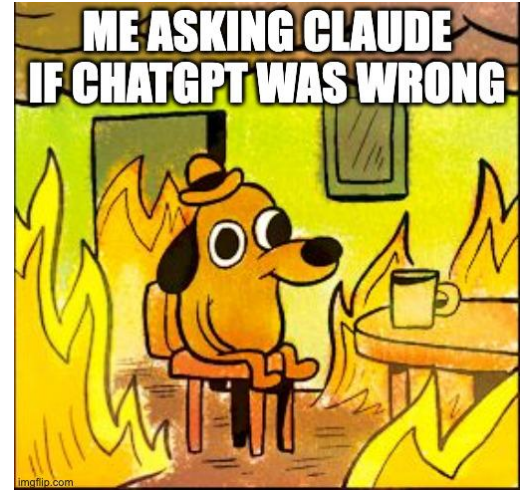
**CONTEXT
FORMATTED LIKE
TRAINING DATA**



Meta Testing: AI as Judge

AI-as-judge was a breakthrough for me

Code can often be written in many different but equally valid ways. So even when Zeta's output differs from our expected answer, it might still be doing exactly what we want, just taking a different path to get there. This makes traditional unit testing approaches particularly challenging when working with LLMs



Lessons

Building in "an evolving space" taught me that the best approach isn't always clear, and that's okay

Rapid prototyping and good feedback loops matter more than having the perfect plan upfront.

I USE NEOVIM NOW

